1 Orbit and Clock Product Evaluation

Every processing mode in the engine — post-processed PPP, PPP-RTK, the baseline modes, and the real-time streaming engine — shares one deterministic satellite *backbone*: a position $\boldsymbol{r}^i$, velocity $\dot{\boldsymbol{r}}^i$, and clock bias δt^i for each satellite at each query epoch. This backbone is evaluated *before* any state-space refinement. In post-processing it is the final word; in the live engine it is the broadcast baseline that the SSR overlay (the SSR corrections document) refines by a RAC orbit delta and a scalar clock delta. This document derives the backbone itself — the two precise interpolation paths and the two broadcast evaluation paths — and the ephemeris-selection logic that feeds them. It deliberately stops at the point where the SSR delta is added: the orbit/clock *overlay* math (RAC frame, IOD gate, freshness) lives in the SSR document and is not re-derived here. The Sagnac (Earth-rotation) range correction that turns $\boldsymbol{r}^i$ at transmit time into a consistent line-of-sight belongs to the observation-physics document; the ERP / polar-motion transforms that carry an SP3 sample between the terrestrial and celestial frames belong to the reference-frames document.

The backbone splits cleanly into a *precise* path and a *broadcast* path. The precise path consumes tabulated SP3 orbit samples and CLK clock samples (IGS Final / Rapid products) and *interpolates*; it is the post-processing default and is centimetre-to-millimetre grade. The broadcast path evaluates a Keplerian (or, for GLONASS, a numerically integrated PZ-90) ephemeris analytically from the navigation message; it is metre-grade on its own and is the substrate the live SSR stream corrects. On the default path the two paths do not mix within a satellite at a given epoch: a precise product, when loaded and spanning the query, wins; otherwise the engine falls through to the broadcast evaluation (`src/core/model/ModelGeometry.cpp`, `src/core/corrections/GnssCorrections.cpp`). The resolution is per quantity, however, not structurally coupled: the real-time RTB-4 gate `realtime.use_precise_orbits` splits the orbit and clock sources independently (`GnssConfig::realtimeUsesLiveOrbitSource` vs `realtimeUsesLiveOrbitClockSource`, `ProductParser::loadProducts(bool, bool)`), so an armed "ultra-rapid orbits + network clocks" config evaluates one satellite at one epoch with a precise interpolated orbit *and* a broadcast+SSR clock. The flag defaults false — the orbit gate then returns exactly the clock gate, byte-identical — so the no-mixing statement holds for every non-real-time mode and for the default real-time path.

1.1 Precise orbit: derotated Neville interpolation

SP3 files tabulate the satellite position in the ECEF frame on a coarse grid (typically 900 s for IGS Final, 300 s for Rapid). The ECEF trajectory is *not* a smooth low-degree polynomial: it carries the sidereal-frequency oscillation of the rotating frame relative to the satellite's near-inertial orbit, which a degree-10 fit cannot capture to better than a few millimetres near the SP3 epoch boundaries. The engine therefore *derotates* each node into a common near-inertial frame anchored at the query epoch before interpolating (`src/core/model/ModelGeometry.cpp`, following the RTKLIB / cssrlib convention [4]). For a node sampled at t_i and a query at t_q, with the Earth-rotation rate $\omega_\oplus = 7.2921151467 \times 10^{-5}$ rad/s (`Constants::EARTH_ANGULAR_VELOCITY`), the in-plane coordinates are rotated by the angle the frame swept between the two epochs:

$$\boldsymbol{r}^i_{\mathrm{I}}(t_i) = \boldsymbol{R}_z(\omega_\oplus\,(t_i - t_q))\;\boldsymbol{r}^i(t_i), \qquad \boldsymbol{R}_z(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{1}$$

Concretely, with $\alpha_i = \omega_\oplus\,(t_i - t_q)$ the derotated in-plane components written to the interpolation arrays are $\tilde{x}_i = \cos\alpha_i\, x_i - \sin\alpha_i\, y_i$ and $\tilde{y}_i = \sin\alpha_i\, x_i + \cos\alpha_i\, y_i$, with z untouched (`ModelGeometry.cpp`). Because the rotation angle is exactly zero at $t_i = t_q$, the interpolated value at the query is *already* the ECEF position at t_q — no post-rotation is needed.

Bracket-centred window. The node window is the decisive design choice for position *continuity*. The engine selects an IGS_POINTS = 11-node window centred on the *bracket* $[t_a, t_{a+1}]$ that contains t_q, not the 11 nodes nearest to t_q (`ModelGeometry.cpp`). The distinction is subtle but operationally important: a nearest-N rule swaps a node at the *midpoint* between SP3 epochs, where the two candidate polynomials disagree by a few millimetres, producing the well-known synchronous-across-constellation 15-minute sawtooth in PPP position. A bracket-centred rule instead swaps the window exactly when t_q crosses a node t_{a+1}; at that instant t_{a+1} is shared by both windows, so the Neville value is identical on either side and the interpolated orbit is continuous across the swap. A gap guard re-picks a narrower MIN_POINTS = 9-node window and rejects the satellite if the residual span still exceeds the fit interval (thresholds in `ModelGeometry.cpp`; tuning values cited, not reproduced). The "fit interval" is not read from the product header but computed as the *median* of the leading consecutive-node spacings (`ModelGeometry.cpp`); this is deliberate, because a multi-file SP3 merge can interleave a finer margin file's off-grid nodes into a coarser arc (e.g. a 5-minute ultra-rapid file concatenated with a 15-minute one — the 1 s dedup window correctly keeps them as non-duplicates, so the merged series is non-uniformly sampled and `ProductParser` warns on the mixed-rate concat). The median grid gives a stable MAX_SPAN guard on such a series where the nominal grid would be ill-defined.

Neville value and derivative. Within the window the engine runs Neville's algorithm simultaneously for the value and its analytic derivative (`src/core/math/Interpolate.cpp`, `Interpolate::nevilleInterpDeriv`). To avoid catastrophic cancellation, the node abscissae are shifted to $s_i = t_i - t_q$ (raw continuous GPS seconds are $\sim 1.36 \times 10^8$, so an un-shifted difference loses ~ 8 digits); the polynomial value and derivative are both invariant under this unit-Jacobian shift. With the tableau column P_i for the value and D_i for the derivative, the coupled recurrence over degree j (evaluating at the shifted query $s_q = 0$) is

$$P_{i,j} = \frac{(s_q - s_{i+j})\,P_{i,j-1} - (s_q - s_i)\,P_{i+1,j-1}}{s_i - s_{i+j}}, \tag{2}$$

$$D_{i,j} = \frac{(s_q - s_{i+j})\,D_{i,j-1} + P_{i,j-1} - (s_q - s_i)\,D_{i+1,j-1} - P_{i+1,j-1}}{s_i - s_{i+j}}, \tag{3}$$

seeded by $P_{i,0} = y_i$ and $D_{i,0} = 0$; the converged scalars are $P_{0,n-1}$ and $D_{0,n-1}$ (`Interpolate.cpp`). The derivative recurrence (3) is exactly $\partial/\partial s_q$ of the value recurrence (2), so no second pass or finite difference is required.

Inertial-to-ECEF velocity. The Neville derivative is taken on the *derotated* channels, so it is the velocity in the frame fixed at t_q — a snapshot inertial frame. Downstream consumers (Doppler line-of-sight rate, orbit-normal attitude, the relativistic $\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i$ term) require the ECEF velocity, recovered by removing the frame rotation $\dot{\boldsymbol{r}}^i_{\mathrm{ECEF}} = \dot{\boldsymbol{r}}^i_{\mathrm{I}} - \boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i$. For $\boldsymbol{\omega}_\oplus = (0, 0, \omega_\oplus)$ this is the

boxed transform applied in `ModelGeometry.cpp`:

$$\boxed{\dot{\boldsymbol{r}}^{i}_{\mathrm{ECEF}} = \begin{bmatrix} v_{\mathrm{I},x} + \omega_{\oplus}\, \boldsymbol{r}^{i}_{y} \\ v_{\mathrm{I},y} - \omega_{\oplus}\, \boldsymbol{r}^{i}_{x} \\ v_{\mathrm{I},z} \end{bmatrix}}. \tag{4}$$

The relativistic clock term is invariant to this conversion, since $\boldsymbol{r}^{i} \cdot (\boldsymbol{\omega}_{\oplus} \times \boldsymbol{r}^{i}) = 0$; only the Doppler and cross-product attitude paths see the difference (`ModelGeometry.cpp`).

1.2 Precise orbit: sidereal-trigonometric LSQ fit

A config-gated alternative (`config.enableTrigSp3Interp()`) replaces the Neville polynomial with an over-determined least-squares fit on a *sidereal-trigonometric basis*, run on the identical derotated channels and time base (`ModelGeometry.cpp`; `Interpolate::trigSvdInterp`). The basis is a constant plus four sine/cosine harmonic pairs of the fundamental sidereal angular rate — nine terms in all:

$$\boldsymbol{r}^{i}_{\mathrm{c}}(\tau) = c_0 + \sum_{f=1}^{4} \big[c_{2f-1} \sin(f\,\Omega_0\,\tau) + c_{2f} \cos(f\,\Omega_0\,\tau) \big], \qquad \tau = t - t_0, \tag{5}$$

where t_0 is the first node (the constant term absorbs the offset, the sin/cos pairs the phase, so the fitted curve is invariant to this choice). The fundamental rate is the angular frequency of the *mean sidereal day*: with the sidereal-day ratio $\kappa = 0.99726956634$ and a nominal solar day of 86400 s,

$$\boxed{\Omega_0 = \frac{2\pi}{\kappa \cdot 86400\ \mathrm{s}} \quad [\mathrm{rad/s}], \qquad \kappa = 0.99726956634,} \tag{6}$$

expressed *per second* so the basis argument τ stays in the caller's seconds and the analytic derivative falls out per second (`Interpolate.cpp`; M-PAGES uses rad/day and divides the velocity by 86400, numerically identical). The constant $\kappa = 0.99726956634$ is the IERS mean sidereal-to-solar day ratio and *is* the physics of the model, so it is reproduced here with its source line.

The fit is solved by Eigen's `JacobiSVD` with a relative singular-value threshold of 10^{-15}, which yields the minimum-norm least-squares solution — the Numerical-Recipes `svbksb` convention M-PAGES' `svdfit` uses [3]. Per-node weights derive from the SP3 `++` accuracy codes; because those are a single value per satellite, the weight set is uniform and the fit is effectively unweighted, matching M-PAGES. The analytic value and derivative at the query $\tau_q = t_q - t_0$ are

$$\boldsymbol{r}^{i}_{\mathrm{c}}(\tau_q) = c_0 + \sum_{f=1}^{4} \big[c_{2f-1} \sin(f\Omega_0\tau_q) + c_{2f} \cos(f\Omega_0\tau_q) \big], \tag{7}$$

$$\dot{\boldsymbol{r}}^{i}{}_{\mathrm{c}}(\tau_q) = \sum_{f=1}^{4} (f\Omega_0) \big[c_{2f-1} \cos(f\Omega_0\tau_q) - c_{2f} \sin(f\Omega_0\tau_q) \big], \tag{8}$$

with the per-harmonic angular weight $w_f = f\,\Omega_0$ (`Interpolate.cpp`). Crucially this is an *over-determined smoother*: the window is up to 11 nodes against 9 basis terms, so for $N > 9$ the fitted curve does *not* pass through the nodes — a deliberate divergence from the M-PAGES 9-point exact fit, kept so the trig branch reuses the existing derotated window verbatim. The same inertial-to-ECEF velocity conversion (4) is then applied to $\dot{\boldsymbol{r}}^{i}{}_{\mathrm{c}}$.

1.3 Precise clock interpolation

The CLK product tabulates the scalar satellite clock bias δt^i on its own grid (commonly 30 s). Because clock noise is process-driven (a near-Brownian random walk in bias, with occasional rate discontinuities), the engine offers three interpolation modes selected by `config.getSatelliteClockInterpolationMode()` (`src/core/corrections/GnssCorrections.cpp`). All three are bracket-centred for the same continuity reason as the orbit window: the sliding window must swap on a shared node, never at a between-epoch midpoint.

(a) Linear (MMSE for Brownian motion). The default is a two-node linear interpolation across the bracket $[t_0, t_1]$ containing t_q:

$$\delta t^i(t_q) = \delta t_0^i + \left(\delta t_1^i - \delta t_0^i\right) \frac{t_q - t_0}{t_1 - t_0}, \tag{9}$$

which is the minimum-mean-square-error estimator for a Wiener process sampled at the two bracketing epochs. Interpolation across a gap larger than a guard threshold (in seconds) is rejected outright rather than fabricated (`GnssCorrections.cpp`; gap value cited, not reproduced).

(b) Rate-anchored weighted LSQ. The `LsqSatClock` mode fits a local line $\delta t^i(t) \approx a + b\,(t - t_q)$ over an $\sim$11-node window by weighted least squares, with the centring shift $s_k = t_k - t_q$ making the fitted value at the query equal to the intercept a directly. The key refinement is that the CLK product may also carry a clock *rate* m_j at the coarse 5-minute epochs; each rate is added as an *explicit observation of the slope*, $m_j = b + \mu_j$. Since a value equation $y_k = a + b\,s_k + \nu_k$ has $\partial/\partial a = 1$, $\partial/\partial b = s_k$ while a rate equation $m_j = b + \mu_j$ has $\partial/\partial a = 0$, $\partial/\partial b = 1$, the rate observations contribute to the slope row only. With value weights $w_k^{(y)} = 1/\sigma_{y,k}^2$ and rate weights $w_j^{(m)} = 1/\sigma_{m,j}^2$, the 2×2 weighted normal equations are

$$\boxed{\begin{bmatrix} \sum_k w_k^{(y)} & \sum_k w_k^{(y)} s_k \\ \sum_k w_k^{(y)} s_k & \sum_k w_k^{(y)} s_k^2 \;+\; \sum_j w_j^{(m)} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum_k w_k^{(y)} y_k \\ \sum_k w_k^{(y)} s_k y_k \;+\; \sum_j w_j^{(m)} m_j \end{bmatrix},} \tag{10}$$

solved in closed form for the intercept

$$a = \frac{\left(S_{wxx} + S_m\right) S_{wy} - S_{wx}\left(S_{wxy} + S_{my}\right)}{S_w\left(S_{wxx} + S_m\right) - S_{wx}^2}, \tag{11}$$

with the running sums named exactly as in `GnssCorrections.cpp` ($S_w = \sum w_k^{(y)}$, $S_{wx} = \sum w_k^{(y)} s_k$, $S_{wxx} = \sum w_k^{(y)} s_k^2$, $S_{wy} = \sum w_k^{(y)} y_k$, $S_{wxy} = \sum w_k^{(y)} s_k y_k$, $S_m = \sum w_j^{(m)}$, $S_{my} = \sum w_j^{(m)} m_j$), and the determinant in the denominator guarding a degenerate (coincident-time) window with a linear fallback. The estimate $\delta t^i(t_q) = a$ is returned. Because the fit smooths $\sim N$ value samples instead of touching two, the observation-noise component of the estimate at the bracket midpoint drops by $\sim\sqrt{N/2}$ relative to plain linear (9) — material in the seconds-to-minutes regime where the CLK observation noise dominates the per-sample process noise. The cost is a loss of exact-node passage: the smoother introduces millimetre-level discontinuities at the clock epochs when the window slides.

(c) Barycentric Lagrange. For high-rate (1 Hz) products a barycentric-form Lagrange interpolation over a LAGRANGE_POINTS = 9-node window is available (`GnssCorrections.cpp`; `Interpolate::barycentricWeights` / `barycentricEval`). The barycentric form

$$\delta t^i(t_q) = \sum_{i=1}^{n} \frac{w_i}{t_q - t_i}\,\delta t_i^i \;\Bigg/\; \sum_{i=1}^{n} \frac{w_i}{t_q - t_i}, \qquad w_i = \prod_{k\neq i} \frac{1}{t_i - t_k}, \tag{12}$$

is the numerically stable evaluation of the degree-$(n-1)$ Lagrange polynomial [1]; an extrapolation guard falls back to two-node linear when t_q lies outside the node span. This mode is *not* recommended for standard 30 s IGS Final clocks — the polynomial oscillates between the sparse samples — and is retained only for genuinely high-rate input.

1.4 Broadcast Keplerian evaluation

When no precise product spans the query, GPS, Galileo, BeiDou and QZSS are evaluated from their Keplerian broadcast ephemeris (`src/core/products/LiveEphemerisStore.cpp`, `keplerPosition`). This is the substrate the live SSR stream corrects. Each system carries its own gravitational parameter μ and Earth-rotation rate $\omega_\oplus$ (`LiveEphemerisStore.cpp`): GPS / QZSS use $\mu = 3.986005 \times 10^{14}\,\mathrm{m^3/s^2}$, $\omega_\oplus = 7.2921151467 \times 10^{-5}\,\mathrm{rad/s}$; Galileo / BeiDou use $\mu = 3.986004418 \times 10^{14}\,\mathrm{m^3/s^2}$, with BeiDou taking the slightly different $\omega_\oplus = 7.292115 \times 10^{-5}\,\mathrm{rad/s}$. BeiDou is evaluated entirely on its native BeiDou Time (BDT) scale — BDT week 0 aligns with GPS week 1356 and BDT = GPS − 14 s — with both the query epoch and the stored t_{oe} kept on BDT so the node-rotation term below uses the BDT seconds-of-week directly (`LiveEphemerisStore.cpp`).

With the semi-major axis $a = (\sqrt{a})^2$ and the time-from-ephemeris $t_k = t - t_{oe}$ reduced into $[-302400, +302400]$ s (week rollover), the corrected mean motion adds the broadcast δn to the Keplerian value, and the mean anomaly propagates linearly:

$$n = \sqrt{\frac{\mu}{a^3}} + \delta n, \qquad M = M_0 + n\,t_k. \tag{13}$$

Kepler's equation $M = E - e\sin E$ is solved by Newton iteration (up to 15 steps, convergence at $|\Delta E| < 10^{-13}$):

$$E_{k+1} = E_k + \frac{M - \big(E_k - e\sin E_k\big)}{1 - e\cos E_k}. \tag{14}$$

The true anomaly and the argument of latitude follow, then the broadcast second-harmonic corrections refine the argument of latitude u, the orbital radius r, and the inclination i:

$$\nu = \operatorname{atan2}\big(\sqrt{1-e^2}\,\sin E,\ \cos E - e\big), \qquad \phi = \nu + \omega, \tag{15}$$

$$\begin{aligned} u &= \phi + c_{us}\sin 2\phi + c_{uc}\cos 2\phi, \\ r &= a\big(1 - e\cos E\big) + c_{rs}\sin 2\phi + c_{rc}\cos 2\phi, \\ i &= i_0 + i\,t_k + c_{is}\sin 2\phi + c_{ic}\cos 2\phi. \end{aligned} \tag{16}$$

The corrected longitude of the ascending node folds in the node rate *relative to* the rotating Earth and the fixed node offset at the ephemeris epoch:

$$\boxed{\Omega = \Omega_0 + \big(\dot\Omega - \omega_\oplus\big)\,t_k - \omega_\oplus\,t_{oe},} \tag{17}$$

where the final $-\omega_\oplus t_{oe}$ uses the seconds-of-week of t_{oe} on the native scale (`LiveEphemerisStore.cpp`). With the in-plane position $(x_o, y_o) = (r\cos u,\, r\sin u)$ the ECEF position is the standard rotation through Ω and i:

$$\boldsymbol{r}^i = \begin{bmatrix} x_o\cos\Omega - y_o\cos i\,\sin\Omega \\ x_o\sin\Omega + y_o\cos i\,\cos\Omega \\ y_o\sin i \end{bmatrix}. \tag{18}$$

The BeiDou GEO additional fixed rotation is not applied (GEO satellites are rarely used for PPP-RTK). Velocity is obtained by central finite difference of (18) over $\pm 0.5\,\mathrm{s}$, $\dot{\boldsymbol{r}}^i \approx [\boldsymbol{r}^i(t+\frac{1}{2}) - \boldsymbol{r}^i(t-\frac{1}{2})]/(2\,\Delta t)$ with $\Delta t = 0.5\,\mathrm{s}$, reusing the same analytic position evaluator (`LiveEphemerisStore.cpp`).

1.5 Broadcast GLONASS: PZ-90 RK4 integration

GLONASS does not broadcast Keplerian elements; it broadcasts an ECEF state vector (position, velocity, and a luni-solar acceleration) valid at t_b, and the ICD prescribes numerical integration of the PZ-90 equations of motion to the query epoch [2]. The engine marches a fixed-step fourth-order Runge–Kutta integrator (`src/core/products/LiveEphemerisStore.cpp`, `glonassState`). The per-axis acceleration combines the central gravitational term, the J_2 oblateness perturbation, the centrifugal and Coriolis terms of the rotating frame, and the broadcast luni-solar acceleration held constant over the integration. Writing $r = \|\boldsymbol{r}^i\|$ and the oblateness coefficient

$$\kappa_{J_2} = \tfrac{3}{2}\,J_2\,\frac{\mu\,a_e^2}{r^5}, \qquad J_2 = 1.0826257\times10^{-3}, \tag{19}$$

(with $\mu = 3.9860044\times10^{14}\,\mathrm{m^3/s^2}$, $a_e = 6378136\,\mathrm{m}$, $\omega_\oplus = 7.292115\times10^{-5}\,\mathrm{rad/s}$, all PZ-90 constants at `LiveEphemerisStore.cpp`), the accelerations are

$$\boxed{\begin{aligned} \ddot{x} &= -\frac{\mu}{r^3}x + \kappa_{J_2}\,x\big(1-5\,z^2/r^2\big) + \omega_\oplus^2 x + 2\omega_\oplus\dot{y} + a_x, \\ \ddot{y} &= -\frac{\mu}{r^3}y + \kappa_{J_2}\,y\big(1-5\,z^2/r^2\big) + \omega_\oplus^2 y - 2\omega_\oplus\dot{x} + a_y, \\ \ddot{z} &= -\frac{\mu}{r^3}z + \kappa_{J_2}\,z\big(3-5\,z^2/r^2\big) + a_z, \end{aligned}} \tag{20}$$

where (a_x, a_y, a_z) is the broadcast luni-solar acceleration. Note the distinct numerator on the z-axis oblateness term ($3 - 5z^2/r^2$ versus $1 - 5z^2/r^2$ in-plane), and that the centrifugal $\omega_\oplus^2$ and Coriolis $2\omega_\oplus$ terms appear only on the in-plane axes, as required in the PZ-90 rotating frame. The integrator steps with a fixed maximum $h = 30\,\mathrm{s}$, signed toward the query, clamping the final partial step so the march lands exactly on t_q; each step is the classical RK4 update $y \leftarrow y + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ (`LiveEphemerisStore.cpp`).

1.6 Broadcast satellite clock

The broadcast satellite clock for the Keplerian systems is the standard second-order polynomial in the time-from-clock-epoch $t - t_{oc}$ (on the native scale, with BeiDou shifted to BDT exactly as the orbit node term is):

$$\boxed{\delta t^i = a_{f0} + a_{f1}\,(t - t_{oc}) + a_{f2}\,(t - t_{oc})^2.} \tag{21}$$

GLONASS instead uses its two-term broadcast model with the 1020-message τ_n (DF123) and γ_n (DF120) about $t_b = t_{oe}$:

$$\delta t^i = -\tau_n + \gamma_n\,(t - t_b) \tag{22}$$

(`LiveEphemerisStore.cpp`). This path is exercised only where GLONASS is excluded from `active_systems` — the shipped real-time replay configs, which is the context this store serves; the default `active_systems` and the post-processing configs both enable GLONASS, so the path is not universally dead code.

The precise-clock-baseline convention. A design decision worth stating explicitly: the broadcast clock (21) *deliberately omits* both the periodic relativistic eccentricity term and the group delay (TGD/BGD). This is not an oversight — it is what makes the broadcast clock a clean drop-in for the SSR scalar clock delta. The IGS-RTS clock correction is generated against precisely this bare polynomial, so the engine must *not* pre-apply the periodic relativity or TGD before adding the SSR delta, or the correction would double-count. The periodic relativistic term $-2\,\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i / c^2$ is added later, in the observation-physics path; see the observation-physics document for its derivation. This convention keeps the orbit, the bare clock, and the SSR overlay mutually consistent on one ephemeris baseline.

1.7 Ephemeris selection and IOD bookkeeping

The broadcast paths above presuppose that the *right* ephemeris record has been selected. The live store holds, per satellite, a `current` and an optional `previous` record, and several interlocking rules govern which one feeds the orbit/clock evaluation (`src/core/products/LiveEphemerisStore.cpp`).

Freshest-toe within a source. Within a single source, the record with the newer reference epoch wins: an incoming record whose t_{oe} is older than the held one (a stale re-broadcast) is rejected, while a sub-second jitter or equal-epoch refresh overwrites in place (`LiveEphemerisStore.cpp`). Selection between `current` and `previous` at query time is by *closest* t_{oe} on a common GPS-continuous scale — BeiDou's BDT t_{oe} is lifted to GPS seconds first (`toeGpsContinuous`), or every BeiDou record would read $\sim$1356 weeks stale and be wrongly skipped.

IGS-source authority over receiver nav. When the engine ingests both an IGS broadcast stream and the receiver's own decoded navigation, the IGS record is *authoritative*. The IGS-RTS SSR delta is computed against IGS broadcast ephemeris, so the rover must pair the delta with the *same* IGS broadcast; even at an identical IOD, the receiver's locally assembled orbit carries slightly different fit parameters (a live Galileo I/NAV assembly was observed to diverge by 100 m+), and applying IGS-SSR on top of a receiver ephemeris detonates the single-difference frame. A receiver record therefore never overwrites a current IGS record at an equal-or-older t_{oe}; it is accepted only when it is strictly newer than the IGS record by more than one constellation refresh (1800 s, the GLONASS nominal step), the signature of a *stalled* IGS feed — so a dead IGS stream degrades gracefully to receiver nav rather than stranding the satellite (`LiveEphemerisStore.cpp`).

IODE-change demote and SSR re-referencing. When the IOD changes, the `current` record is demoted to `previous` (rather than discarded), keeping the just-superseded orbit valid across the cutover instant (`LiveEphemerisStore.cpp`). This matters because, across a cutover, the receiver can hold a version one step ahead of the lagging SSR stream. When an SSR delta names a specific IOD, `pickForSsr` re-references: it returns whichever held record (`current` or `previous`) matches the SSR's IOD, so the orbit and clock are evaluated on the exact baseline the correction was computed for, instead of dropping the satellite to broadcast grade on a `current`-vs-SSR mismatch.

The same current/previous retention underwrites a second consumer built on the broadcast clock polynomial (21): `LiveEphemerisStore::ephCutoverShiftMetres` evaluates $c\,(\delta t^{i}_{\text{new}} - \delta t^{i}_{\text{old}})$ across an ephemeris version roll — `computeSatelliteClock` run once on the freshest pick and once pinned to the old IODE — and returns nothing when the old version is no longer retained. It is the product-side half of the network producer's `network.apriori_source = brdc_rts` eph-cutover re-level; that re-level is derived in the Network mode document and is not repeated here.

The GNSS-IOD matching convention. The SSR "GNSS IOD" field (IGS-SSR / RTCM `4076`, IDF012) is an 8-bit quantity whose per-constellation *definition* differs from the raw broadcast IODE stored on the ephemeris. `ssrMatchIode` normalises the held record's IODE into the SSR convention in exactly one place (`LiveEphemerisStore.cpp`):

$$\mathrm{IOD}_{\mathrm{SSR}}(i) = \begin{cases} \mathrm{IODE} & \text{GPS, QZSS (8-bit LNAV, passthrough),} \\ \mathrm{IODnav}\ \&\ \texttt{0xFF} & \text{Galileo (low 8 bits of the 10-bit I/NAV IODnav),} \\ \left\lfloor t_{oe}^{\mathrm{sow}}/720 \right\rfloor \bmod 240 & \text{BeiDou (toe-derived, per IGS-SSR v1.00),} \end{cases} \tag{23}$$

with t_{oe}^{sow} the BDT seconds-of-week. The BeiDou broadcast AODE that the decoder stores in the IODE field is a *different* quantity and would never match the SSR field, so it is deliberately ignored in favour of the toe-derived value. The raw IODE is left intact for the demote bookkeeping above; only the value handed to the SSR gate is normalised here. How that normalised IOD then *gates* the application of the SSR orbit/clock delta — accept the delta only on an exact IOD match, else fall back to broadcast grade — is the subject of the SSR IOD-gate section; this document supplies only the matched quantity it consumes.

References

[1] Jean-Paul Berrut and Lloyd N. Trefethen. "Barycentric Lagrange Interpolation". In: *SIAM Review* 46.3 (2004), pp. 501–517. DOI: 10.1137/S0036144502417715.

[2] *GLONASS Interface Control Document: Navigational Radiosignal in Bands L1, L2 (Edition 5.1)*. Russian Institute of Space Device Engineering. Moscow, 2008.

[3] William H. Press et al. *Numerical Recipes: The Art of Scientific Computing*. 3rd. Cambridge: Cambridge University Press, 2007.

[4] Tomoji Takasu and Akio Yasuda. "Development of the Low-Cost RTK-GPS Receiver with an Open Source Program Package RTKLIB". In: *International Symposium on GPS/GNSS*. Jeju, Korea, 2009, pp. 4–6.